

Architecting Claude Code: The Definitive Runbook

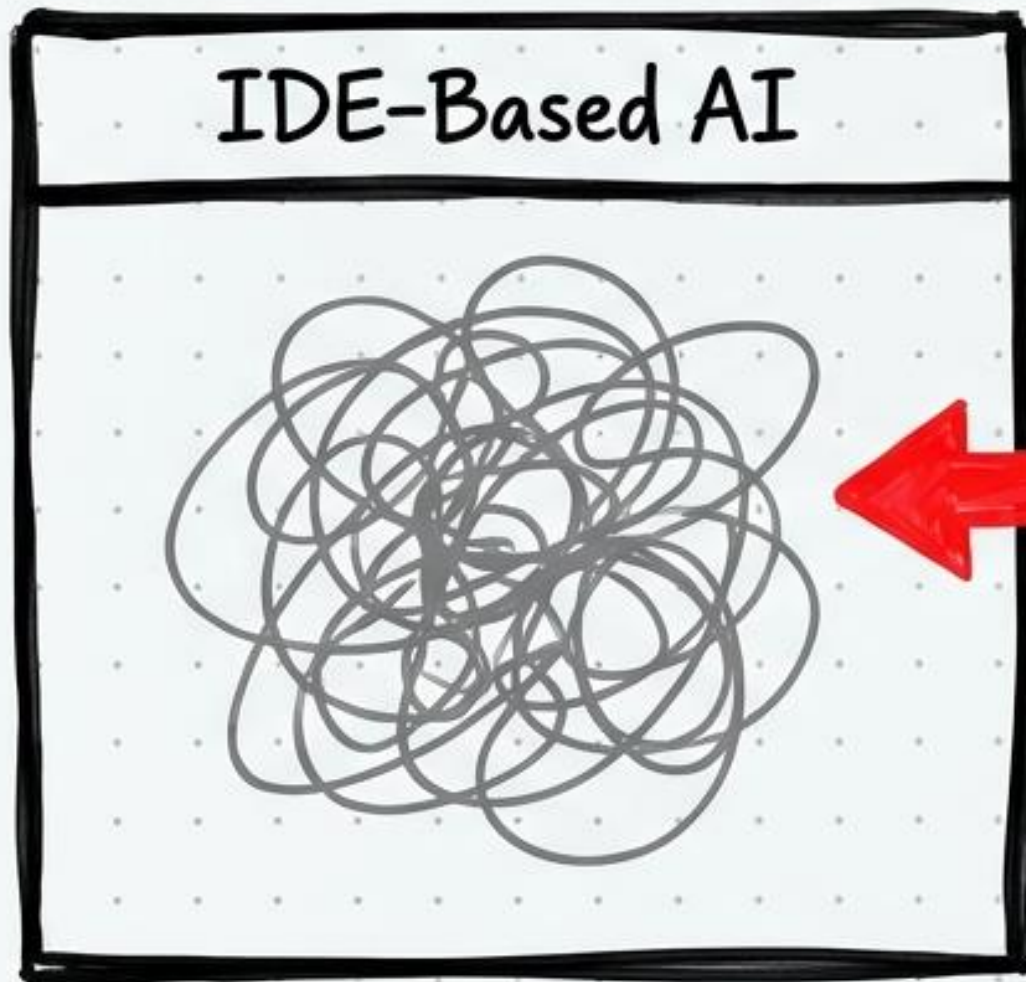
A Principal Engineer's guide to the
5-layer, file-native customization model.

v1.0 | Target: Local Dev Workflows

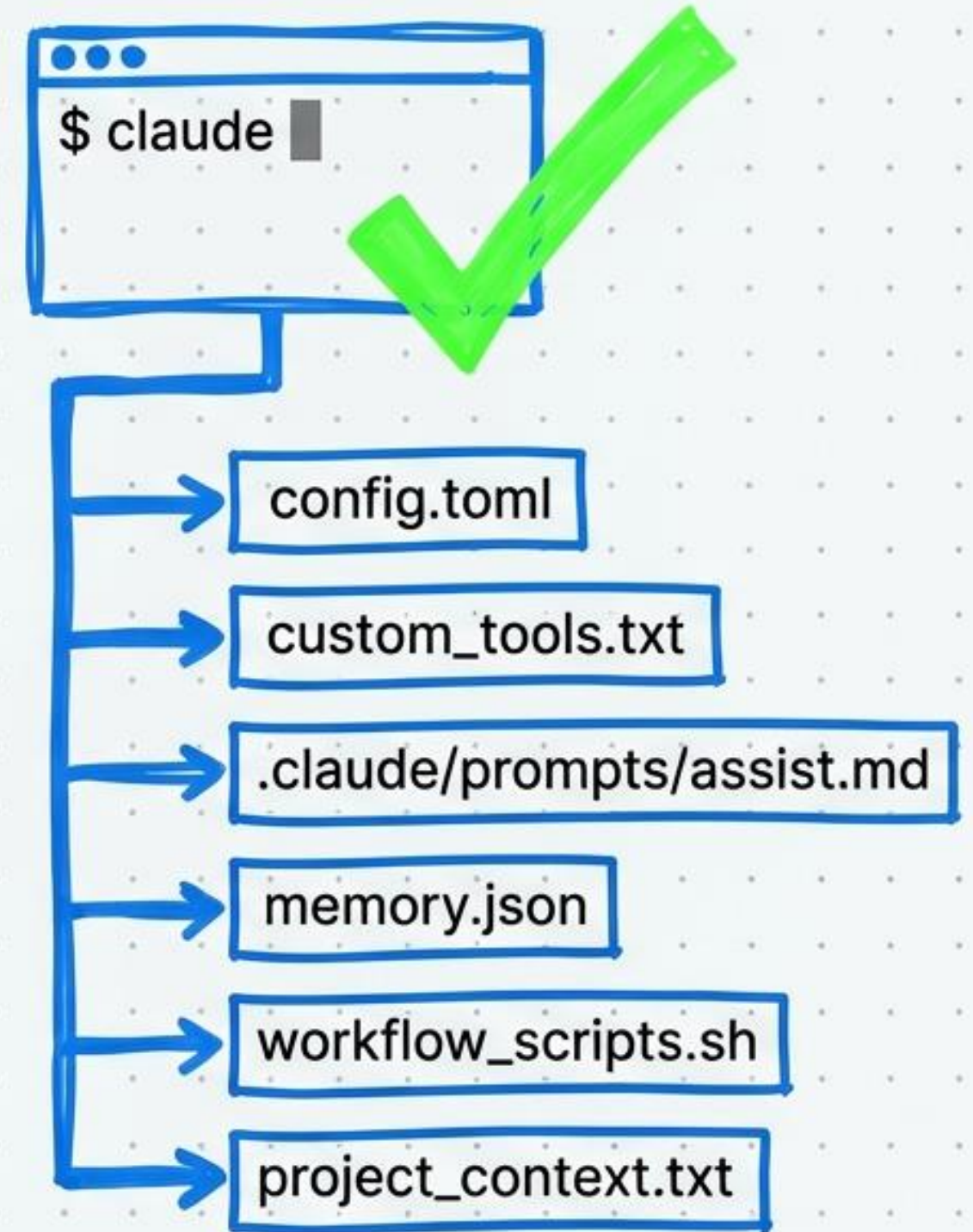


The Philosophy: File-First, Code-Native

- Claude Code operates from the terminal, not a hidden GUI panel.
 - Every customization is a plain text file.
 - Everything is tracked in version control.
- Treat your AI teammate like code.

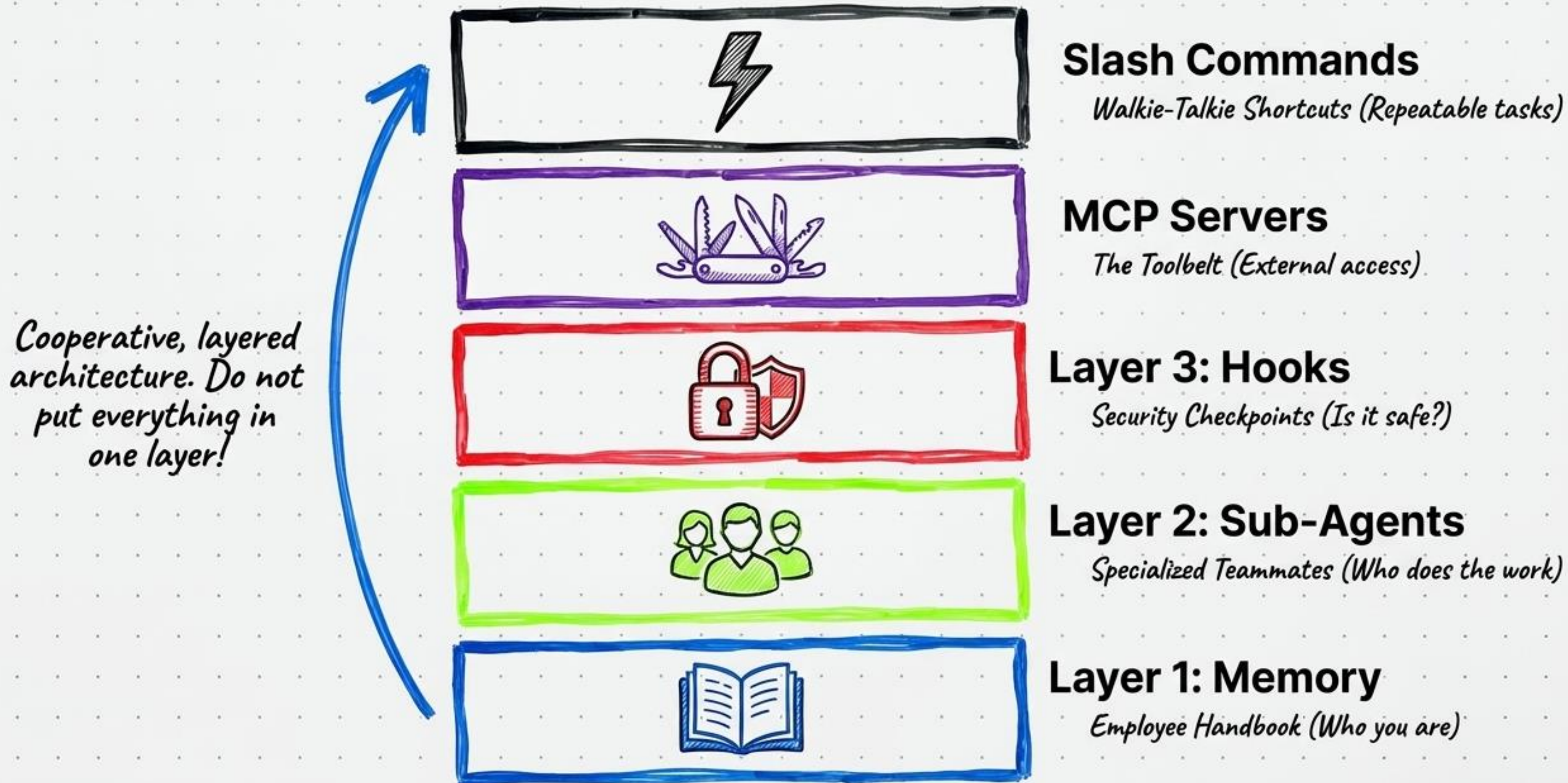


Black Box /
Hidden UI State



Transparency & Control.
Your AI is part of your codebase.

The 5-Layer Customization Architecture



Memory System: The Precedence Funnel

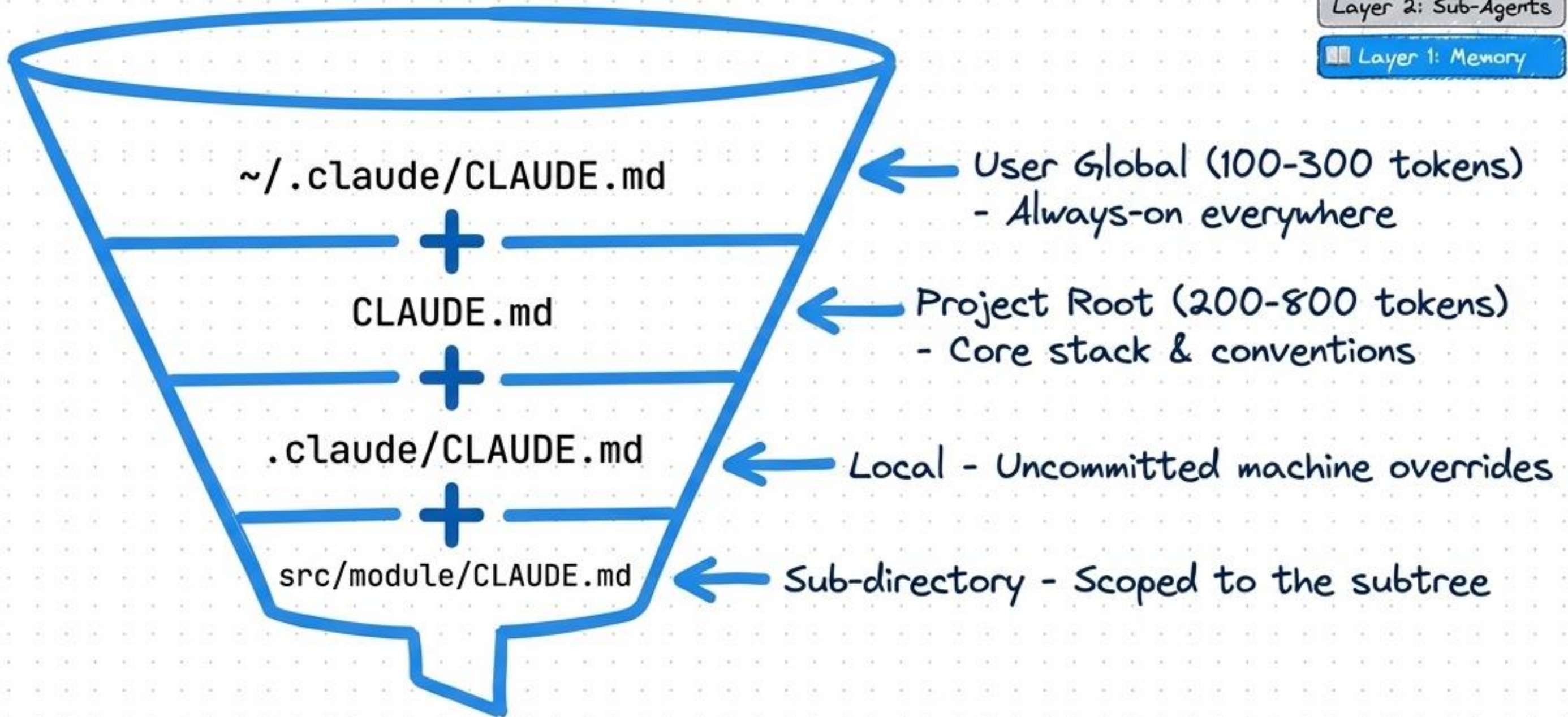
Slash Commands

MCP Servers

Layer 3: Hooks

Layer 2: Sub-Agents

Layer 1: Memory



Redundant rules = context bloat. Never contradict layers.

Sub-Agents: The Power of Isolated Context

Sub-agents do not inherit parent history. This guarantees security, token efficiency, and parallel execution.

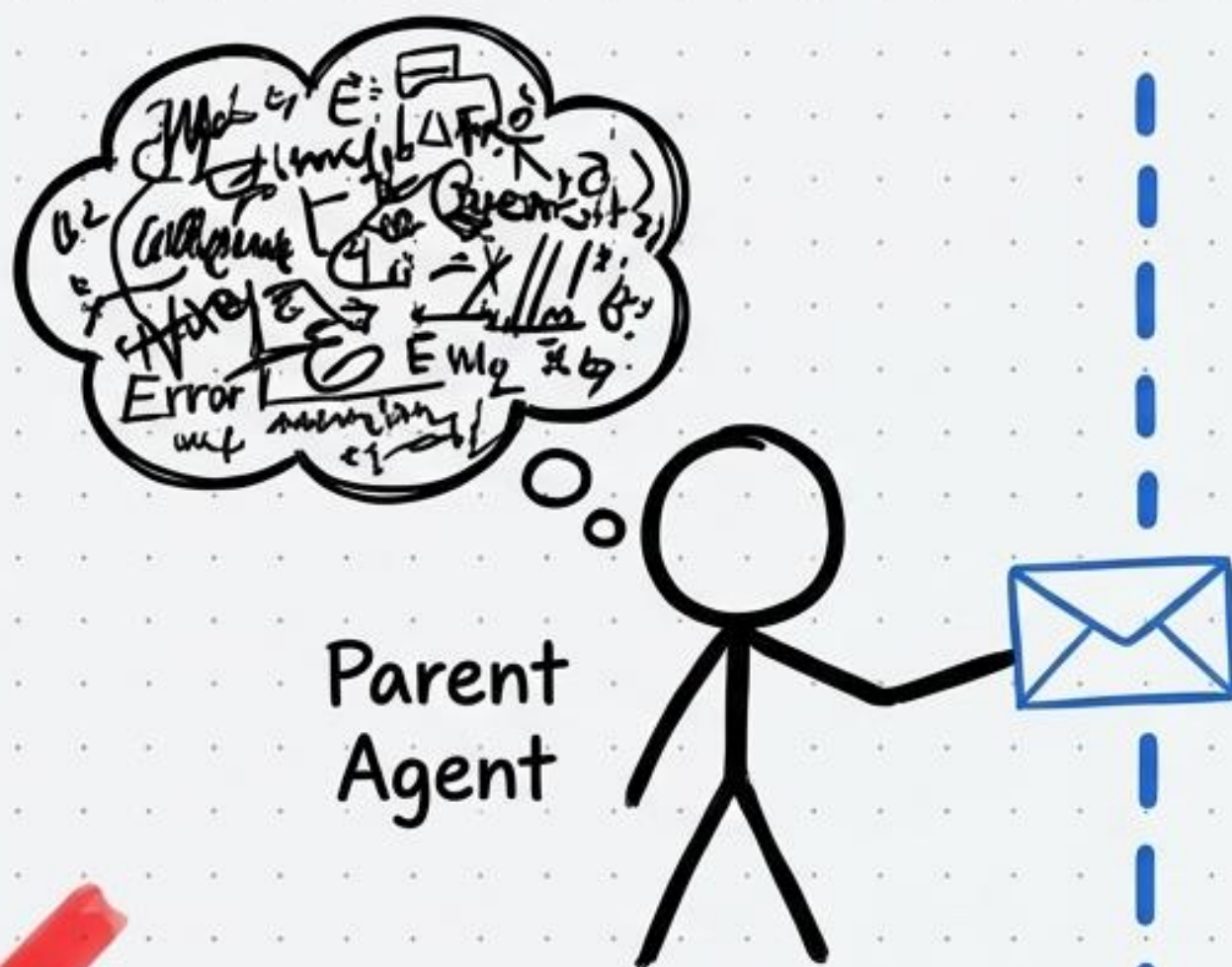
Slash Commands

MCP Servers

Layer 3: Hooks

Layer 2: Sub-Agents

Layer 1: Memory



Isolated Context Sandbox



Single-Level
Hierarchy Only.
No infinite loops.

Sub-Agents: Implementation Rules

Markdown + YAML

```
name: code-reviewer
description: ...
tools: [Read]
```

Storage Location

- Global:
~/.claude/agents/*.md
- Project:
.claude/agents/*.md

CRITICAL: The description is the trigger! Claude reads this to decide when to invoke the agent autonomously. Be explicit.

Slash Commands

MCP Servers

Layer 3: Hooks

Layer 2: Sub-Agents

Layer 1: Memory

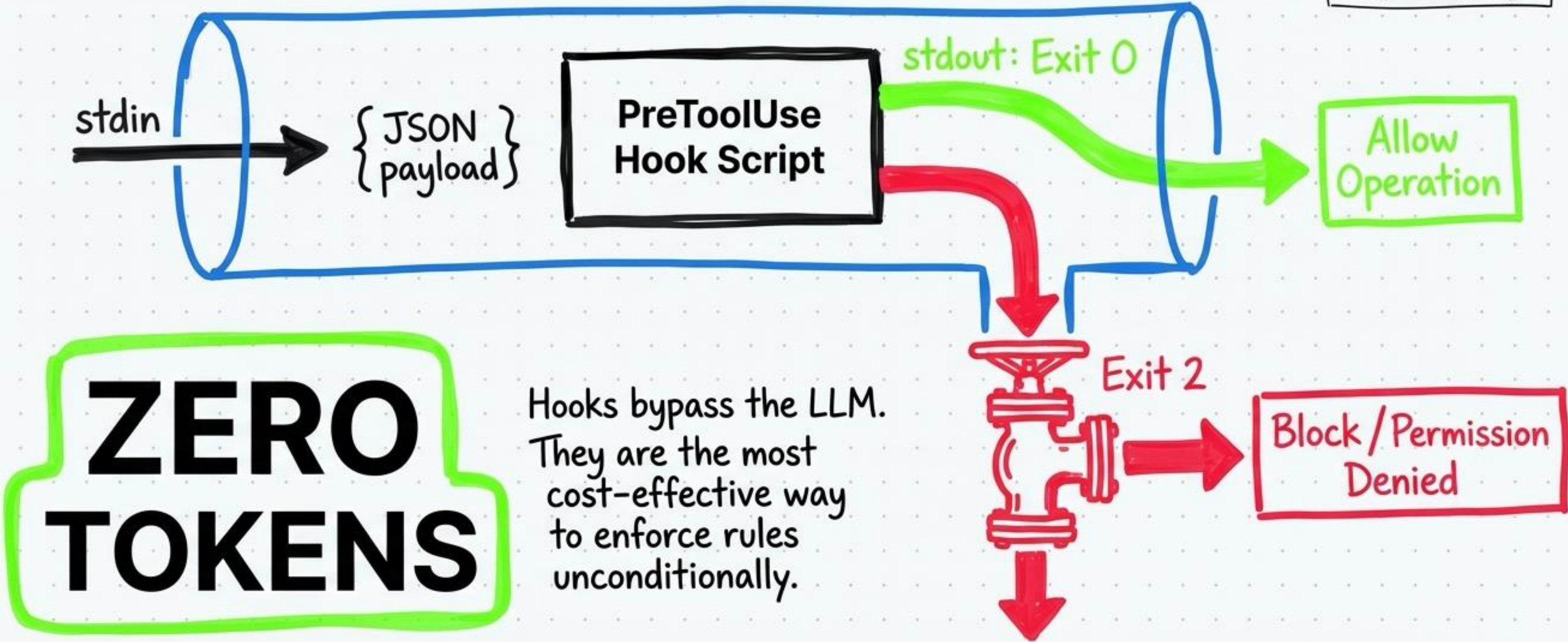
Developer
Writes Code

Reviewer Sub-agent
runs autonomously
until approved

Automated
Quality Loop

Hooks: Deterministic Enforcement

Slash Commands
MCP Servers
Layer 3: Hooks
Layer 2: Sub-Agents
Layer 1: Memory



Practical Application of Hooks

Slash Commands
MCP Servers
Layer 3: Hooks
Layer 2: Sub-Agents
Layer 1: Memory

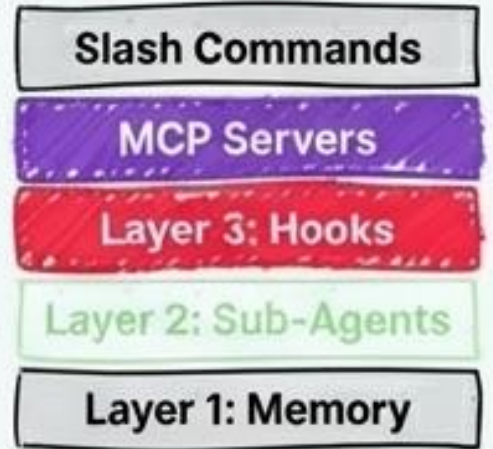
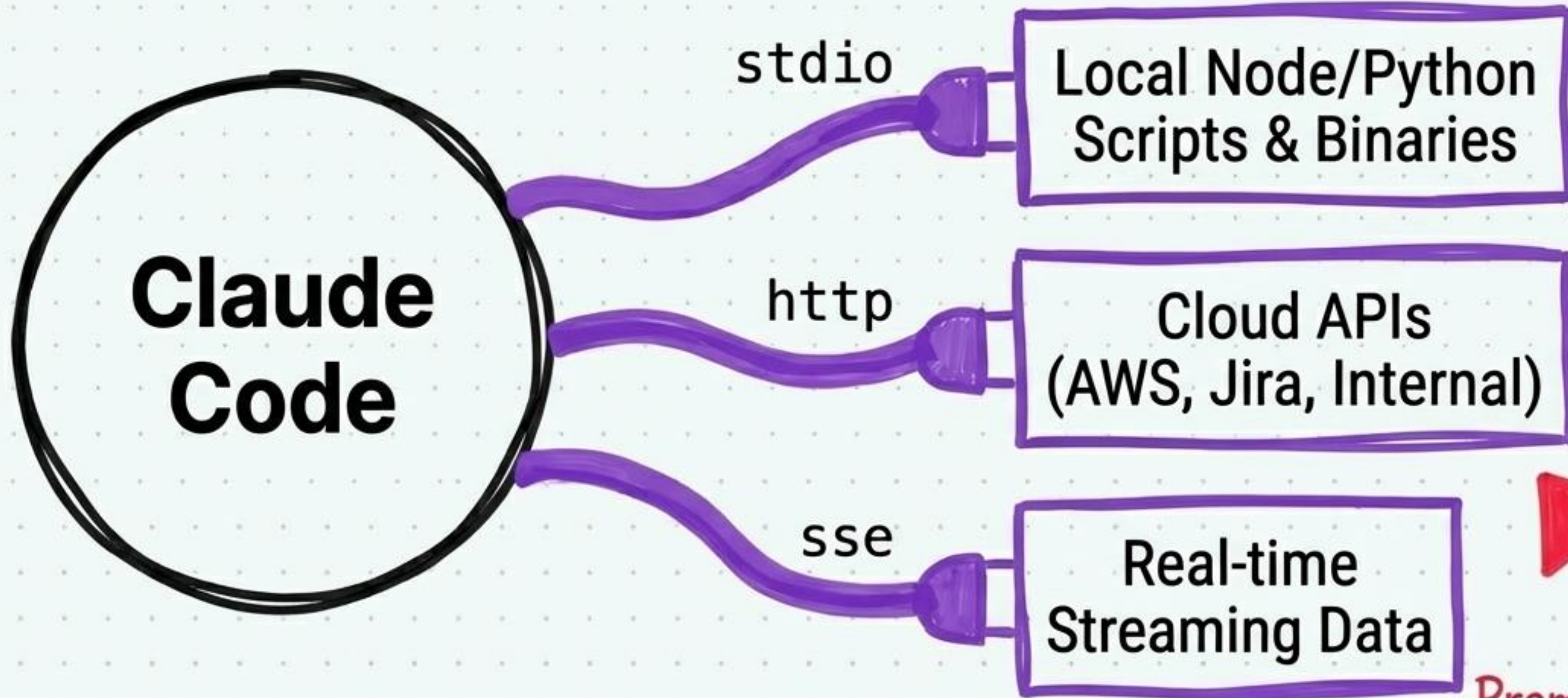
Event Type	Trigger	Use Case
PreToolUse	Before tool call	Block dangerous shell commands (e.g., rm -rf)
PostToolUse	After tool call	Auto-run linter/formatter on save
SubagentStop	Sub-agent finishes	Aggregate results / log output

```
.claude/settings.json  
| "hooks": {  
|   "PreToolUse": "node check-perms.js"  
| }  
|
```



Never use a CLAUDE.md instruction (“Always lint after saving”) for something a Hook can do deterministically.

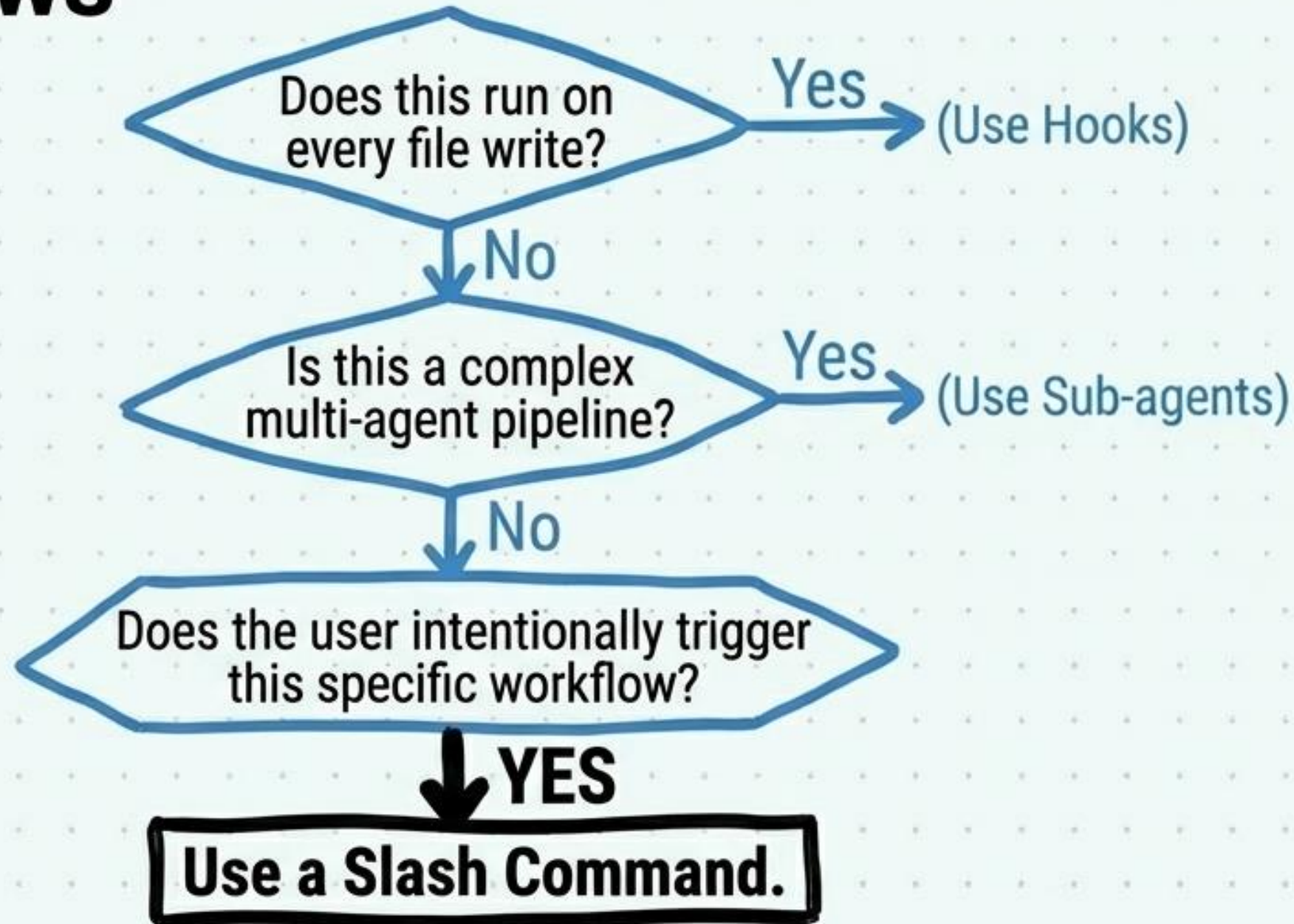
MCP Servers: The External Toolbelt



Configured via ``mcpServers`` key in ``.claude/settings.json``.

Prompt Injection Risk!
Tool descriptions enter the model's context. Audit third-party MCP servers.

Slash Commands: User-Triggered Workflows



Slash Commands

MCP Servers

Layer 3: Hooks

Layer 2: Sub-Agents

Layer 1: Memory



/project:<name>
\$ARGUMENTS

Stored as Markdown in `` . claude/commands/ ``. The ultimate reusable prompt macro for on-demand task execution.

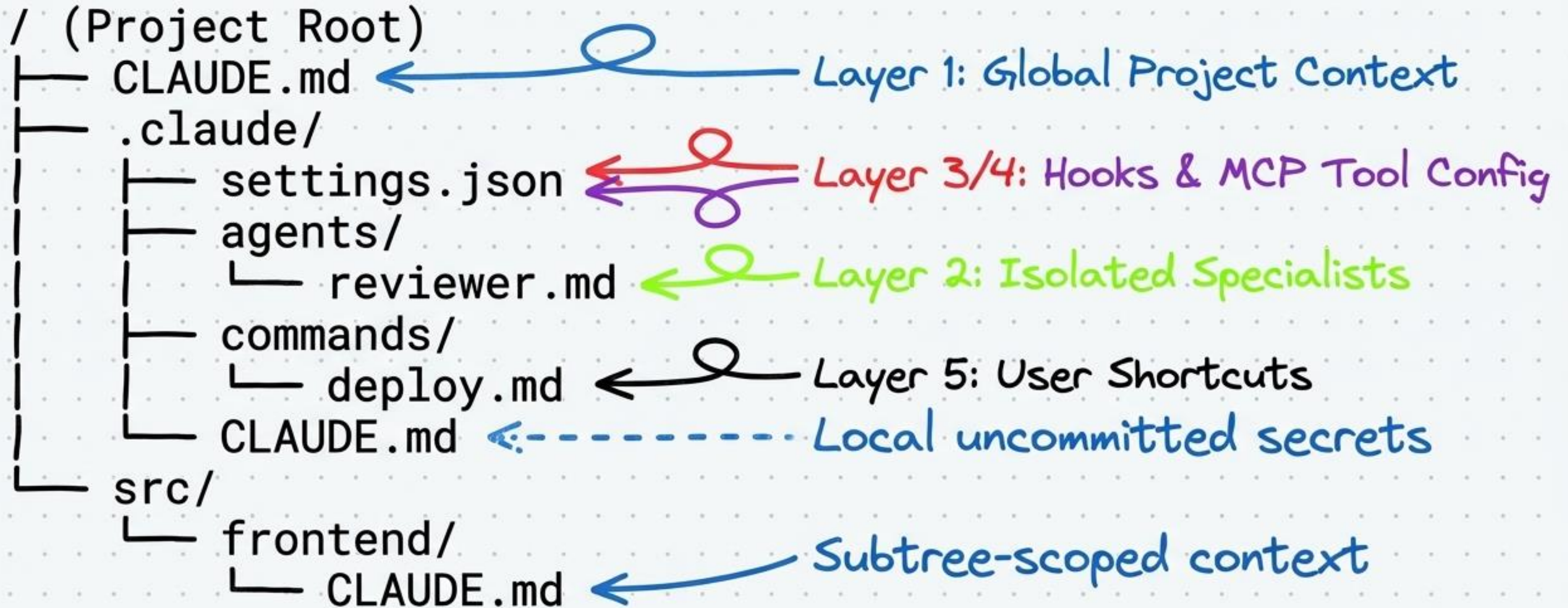
The Diagnostic Matrix: Where Does the Workflow Belong?

Stop putting 15-step debugging workflows in your root CLAUDE.md!

Reserve this strictly for 'Always-On' context.

	Memory	Hooks	Sub-agents	Slash Cmds	MCP Servers
Global Standards & Conventions	✓				
Lint-on-save / Block specific commands		✓			
Automated Parallel Code Review			✓		
User-Triggered Database Migration				✓	
Accessing Internal Jira Tickets					✓

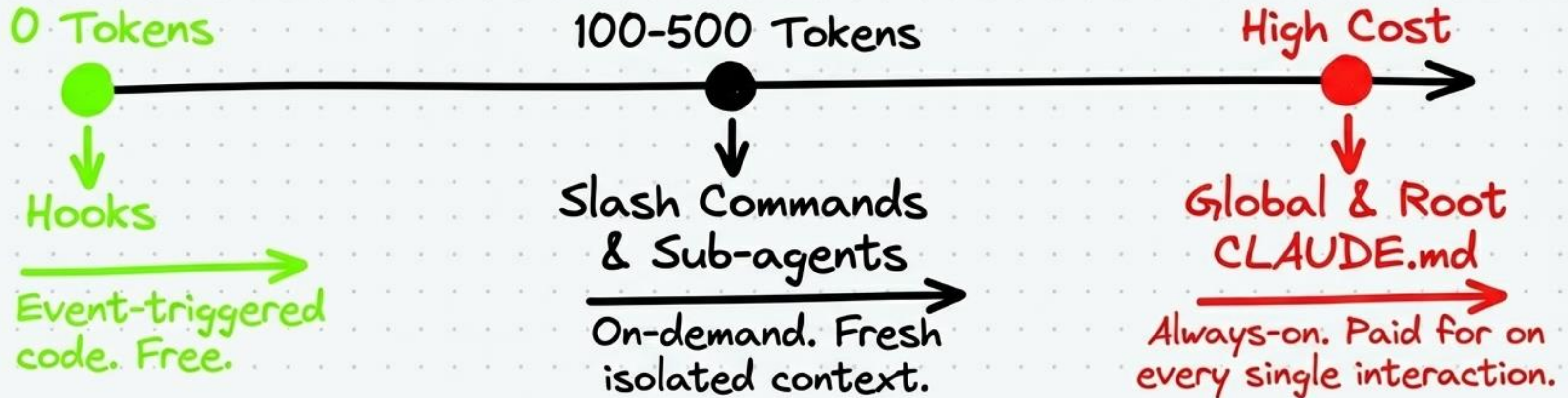
Synthesis: The Complete Project Structure



The Complete Blueprint. Everything in its right place, checked into version control, cooperating seamlessly.

Engineering Constraints: Token Economics

Token Cost Spectrum



The Golden Rule

Never **always-on** what can be on-demand. Sub-agents are cheaper than context accumulation.

Engineering Constraints: Security Posture



Default Deny (.claude/settings.json)

```
!Bash(*)
```

Explicitly enumerate allowed commands.
Deny all shell access by default.



Least Privilege (Sub-agents)

```
disallowedTools: [ExecuteCommand]
```

Reviewers don't need write access.
Restrict tool sets per agent.



Container Isolation

Risky agents (builders, migrators) must run inside containers. Never use 'bypassPermissions' in production.

Architectural Teardown: Claude Code vs. GitHub Copilot

	Claude Code	GitHub Copilot
Environment	Terminal-first	IDE-first
Sub-agent Definition	Markdown + YAML (.claude/agents/)	.agent.md (.github/agents/)
Hooks	Stdin/Stdout Exit Code 2 (Strict Block)	UI permissions toggle
Token Efficiency	Hooks = 0 tokens; Agents strictly isolated	Instructions are always-on

Build for the terminal. Version control
your AI. Architect for reliability.